

OS Notes

Aadesh Lokhande

1. Define an Operating System. Mention its main functions.

An Operating System (OS) is system software that acts as an interface between the user and the computer hardware. It manages all the hardware and software resources of a computer and provides a platform on which application programs can run. Without an operating system, a computer cannot work properly or be used easily by users.

The main functions of an operating system include process management, which controls the execution of programs and allows multitasking. Memory management is another important function; it allocates and deallocates memory to programs as needed. File management helps in creating, storing, organizing, and protecting files on storage devices. Device management controls and coordinates the use of hardware devices like printers, keyboards, and disks. The operating system also provides security by preventing unauthorized access and offering user authentication. Additionally, it offers a user interface, such as a graphical or command-line interface, to help users interact with the computer easily.

2. Distinguish between system software and application software

System software is a type of software that manages and controls the basic operations of a computer. It acts as an interface between the hardware and the user. System software is essential for the computer to function and start properly. Examples of system software include operating systems like Windows, Linux, and macOS, as well as device drivers and utility programs. It usually runs in the background and is not directly used by users for specific tasks.

Application software, on the other hand, is designed to help users perform specific tasks or activities. It depends on system software to run and cannot function without it.

Application software is user-oriented and is used directly by users. Examples include word processors, web browsers, media players, spreadsheet programs, and games. While system software focuses on managing hardware and system resources, application software focuses on completing user tasks efficiently and conveniently.

3. State Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a mechanism that allows different processes running in an operating system to communicate with each other and share data. Since processes are usually independent and have their own memory space, IPC provides a controlled way for them to exchange information and coordinate their actions. IPC is important in multitasking and multiuser systems where many processes run at the same time.

The main purpose of IPC is to enable cooperation between processes, improve system efficiency, and maintain data consistency. Common IPC methods include pipes, message queues, shared memory, and signals. Pipes allow one process to send data to another in a simple way. Message queues enable processes to send and receive messages in an organized manner. Shared memory allows multiple processes to access the same memory area for fast communication. Signals are used to notify processes about events. IPC helps processes work together smoothly without interfering with each other.

4. State fragmentation. Difference between internal and external fragmentation

Fragmentation is a memory management problem that occurs when available memory is not used efficiently. It happens when memory is divided into small pieces, making it difficult to allocate memory to processes even when enough total memory is available. Fragmentation reduces system performance and memory utilization.

Internal fragmentation occurs when a process is allocated more memory than it actually needs. The extra memory inside the allocated block remains unused but cannot be given to other processes. This type of fragmentation usually happens in fixed-size memory allocation systems. For example, if a process needs 18 KB of memory but is given a 20 KB block, the remaining 2 KB is wasted inside the block.

External fragmentation occurs when free memory is scattered in small blocks across the system. Although the total free memory may be sufficient, it is not available in one continuous block. This usually happens in variable-size memory allocation systems. External fragmentation can be reduced using techniques like compaction.

In short, internal fragmentation wastes memory inside allocated blocks, while external fragmentation wastes memory between allocated blocks.

5. Differentiate between process and thread

A process is a program that is currently being executed by the computer. It has its own memory space, system resources, and execution environment. Processes are independent of each other, which means one process cannot directly access the memory of another process. Creating and managing processes requires more system resources, and communication between processes is slower.

A thread is a smaller unit of execution within a process. Multiple threads can exist inside a single process and share the same memory, files, and resources of that process. Threads are lightweight compared to processes and require less time and resources to create and manage. Because threads share memory, communication between them is faster, but it also needs careful handling to avoid errors.

In simple terms, a process is like a separate program running on the system, while a thread is a part of that program that performs a specific task. Processes provide better isolation, whereas threads provide better efficiency and faster execution.

6. Explain OS structure with microkernel approach

The microkernel approach is an operating system structure where only the most essential functions are kept inside the kernel. These basic functions include process management, memory management, and inter-process communication (IPC). Other services such as device drivers, file systems, and system services are moved outside the kernel and run in user mode.

In this approach, the operating system is divided into small, independent modules that communicate with each other using message passing. Since most services run in user space, the kernel remains small and simple. This improves system stability and security because a failure in one service does not crash the entire system.

The microkernel approach also makes the operating system easier to maintain and extend. New services can be added or modified without changing the core kernel. However, frequent communication between user services and the kernel can reduce performance. Examples of microkernel-based operating systems include Mach and MINIX.

7. Differentiate between One-to-One multithreading model and Many-to-Many multithreading model with diagram

In the One-to-One multithreading model, each user-level thread is mapped to a separate kernel-level thread. This means the operating system directly manages every thread. The main advantage of this model is that if one thread is blocked, other threads can continue execution. It also allows true parallelism on multi-core processors. However, creating and managing many kernel threads requires more system resources, which can reduce performance when a large number of threads are used. Examples include Windows and Linux.

In the Many-to-Many multithreading model, many user-level threads are mapped to a smaller or equal number of kernel-level threads. The operating system manages kernel threads, while the user-level thread library manages user threads. This model combines the advantages of both user and kernel threads. It allows better performance and flexibility, as blocking one thread does not block the entire process, and fewer kernel threads reduce overhead. However, it is more complex to implement.

Diagram:

One-to-One Model

User Threads → Kernel Threads

U1 → K1

U2 → K2

U3 → K3

Many-to-Many Model

User Threads → Kernel Threads

U1, U2 → K1

U3, U4 → K2

8. Describe page replacement algorithms (FIFO, LRU, OPTIMAL) with examples

Page replacement algorithms are used in an operating system to decide which memory page should be removed from main memory when a new page needs to be loaded and the memory is full. The goal is to reduce page faults and improve system performance.

FIFO (First In First Out) is the simplest page replacement algorithm. In this method, the page that entered memory first is removed first. For example, if pages 1, 2, and 3 are loaded in this order, page 1 will be replaced when a new page arrives, even if it is still being used.

LRU (Least Recently Used) replaces the page that has not been used for the longest time. It considers recent usage of pages. For example, if pages 1, 2, and 3 are in memory and page 1 was used most recently while page 2 was used long ago, page 2 will be replaced.

OPTIMAL page replacement replaces the page that will not be used for the longest time in the future. For example, if page 3 will not be needed again soon, it is replaced. This algorithm gives the best result but is difficult to implement in real systems.

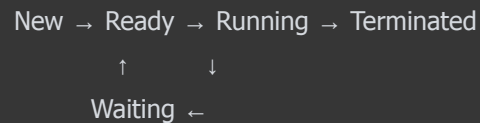
9. Elaborate Process Life Cycle with diagram

The process life cycle describes the different states a process goes through during its execution in an operating system. When a program is started, it first enters the **New** state, where the process is being created. After creation, it moves to the **Ready** state, where it is waiting in the ready queue for CPU allocation.

When the CPU is assigned, the process enters the **Running** state and starts executing instructions. If the process needs input/output operations or has to wait for some event, it moves to the **Waiting (or Blocked)** state. Once the event is completed, it returns to the Ready state.

If the process finishes its execution, it enters the **Terminated** state, where all resources allocated to it are released by the operating system. This cycle helps the OS manage multiple processes efficiently and ensures proper CPU utilization.

Diagram:



11. Discuss about contiguous Memory Allocation in detail

Contiguous memory allocation is a memory management technique in which each process is allocated a single continuous block of memory. In this method, the entire memory is divided into two main parts: one part for the operating system and the remaining part for user processes. When a process arrives, it is loaded into a free block of memory that is large enough to hold it.

There are two main types of contiguous memory allocation: fixed partitioning and variable partitioning. In fixed partitioning, memory is divided into fixed-size partitions, and each process is placed in one partition. This method is simple but can lead to internal fragmentation. In variable partitioning, memory blocks are created according to the size of the process, which reduces internal fragmentation but may cause external fragmentation.

Contiguous memory allocation is easy to implement and manage, and it allows fast access to memory because addresses are continuous. However, it has limitations such as fragmentation and difficulty in allocating memory to large processes when sufficient continuous space is not available. Because of these problems, modern operating systems prefer non-contiguous memory allocation methods like paging and segmentation.

12. Explain Segmentation with Paging

Segmentation with paging is a memory management technique that combines the advantages of both segmentation and paging. In this method, a program is first divided into logical parts called segments, such as code, data, and stack. Each segment represents a meaningful part of the program and can be of different sizes, which helps in better program organization and protection.

Instead of storing each segment as a single continuous block in memory, segmentation with paging further divides each segment into fixed-size pages. These pages are then stored in non-contiguous memory frames. This removes the problem of external fragmentation that occurs in pure segmentation, because pages can be placed anywhere in memory. The operating system uses a segment table and a page table to manage this system. The segment table keeps information about each segment, while the page table maps pages to memory frames. When a process accesses memory, the address is translated using both tables.

Segmentation with paging provides logical memory structure, better memory utilization, and protection. However, it increases system complexity and requires more hardware support for address translation.

15. Describe the Evolution of Operating System in detail

The evolution of operating systems shows how computers developed from simple machines into powerful and user-friendly systems. In the early days, computers did not have operating systems. Programs were loaded and run manually, and only one job could be executed at a time. This stage is known as the serial processing system.

Later, batch operating systems were introduced. In this system, similar jobs were grouped together and executed one after another without user interaction. After that, multiprogramming operating systems were developed, allowing multiple programs to stay in memory at the same time, improving CPU utilization.

Time-sharing operating systems came next, where multiple users could interact with the system simultaneously, each getting a small time slice of the CPU. With the growth of personal computers, single-user and multitasking operating systems like MS-DOS and Windows were developed.

Modern operating systems support multiprocessing, multithreading, distributed systems, and real-time processing. They also provide graphical user interfaces, better security, networking support, and efficient resource management, making computers suitable for a wide range of applications.

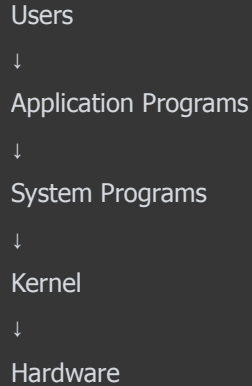
16. Discuss the structure of an Operating System with a neat diagram

The structure of an operating system refers to the way its components are organized and interact with each other. An operating system is designed in layers so that each part performs specific functions and works together efficiently. This structured design helps in easy development, maintenance, and better system performance.

At the lowest level is the hardware, which includes the CPU, memory, and input/output devices. Above the hardware lies the kernel, which is the core of the operating system. The kernel manages process scheduling, memory management, device management, and system calls. On top of the kernel are system programs such as file management, device drivers, and utilities that provide services to users and applications.

The highest layer consists of application programs like word processors, browsers, and media players, which are directly used by users. Users interact with the system through a user interface, such as a command-line interface or graphical user interface.

Diagram:



17. Describe various Operating System services

Operating system services are functions provided by the operating system to make computer usage convenient, efficient, and secure. These services help users and application programs interact with the hardware properly.

One important service is **program execution**, which loads programs into memory, runs them, and terminates them after completion. **Process management** allows the operating system to create, schedule, and control processes, enabling multitasking. **Memory management** is another key service that allocates memory to programs and frees it when no longer needed.

The operating system also provides **file system services**, which help in creating, deleting, reading, writing, and organizing files on storage devices. **Input/Output device management** controls communication between the system and hardware devices like printers, disks, and keyboards through device drivers.

For protection and safety, the OS offers **security services** such as authentication and access control. **Error detection and handling** helps in identifying and correcting system errors. Additionally, the operating system provides a **user interface**, either command-line or graphical, to allow users to interact easily with the computer system.

18. Compare Batch OS, Multiprogrammed OS, and Time-sharing OS

A Batch Operating System is an early type of operating system where jobs with similar requirements are collected in batches and executed one after another without user interaction. Users submit jobs and wait for results later. CPU utilization is low because the CPU may remain idle during input/output operations. It is simple to use but provides no interaction and has long turnaround time.

A Multiprogrammed Operating System allows multiple programs to reside in memory at the same time. When one process is waiting for input/output, the CPU switches to another process, which improves CPU utilization. There is little or no direct user interaction, but system efficiency is much better than batch systems. It mainly focuses on maximizing CPU

usage.

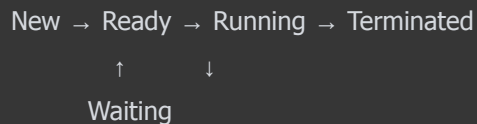
A Time-sharing Operating System is designed to allow multiple users to interact with the system simultaneously. Each process or user is given a small time slice of the CPU in a round-robin manner. This provides quick response time and good user interaction. Time-sharing systems are more complex but are widely used in modern multi-user environments.

19. Define a process and explain the process life cycle with diagram

A process is a program that is currently in execution. It is an active entity that includes the program code, current activity, program counter, registers, and allocated resources such as memory and files. A process is different from a program because a program is only a passive set of instructions stored on disk, while a process represents those instructions being executed by the CPU.

The process life cycle describes the various states a process goes through during its execution. Initially, a process is in the **New** state, where it is being created. It then moves to the **Ready** state, waiting for CPU allocation. When the CPU is assigned, the process enters the **Running** state and starts execution. If the process needs to wait for an input/output operation or some event, it moves to the **Waiting (Blocked)** state. After the event is completed, it returns to the Ready state. Once execution finishes, the process enters the **Terminated** state and releases all resources.

Diagram:



20. Demonstrate the Inter-Process Communication (IPC) mechanisms

Inter-Process Communication (IPC) mechanisms are used by an operating system to allow processes to communicate with each other and synchronize their actions. Since processes run independently and have separate memory spaces, IPC provides safe and controlled methods for data exchange.

One common IPC mechanism is **Pipes**, which allow one process to send data to another in a unidirectional manner. Pipes are mainly used between related processes. **Message Queues** allow processes to send and receive messages in an organized way, where messages are stored in a queue until they are received. This method supports communication between unrelated processes.

Shared Memory is the fastest IPC mechanism. In this method, multiple processes share a common memory area to read and write data. However, synchronization tools like semaphores are required to avoid conflicts. **Semaphores** are used for process synchronization and help control access to shared resources.

Another mechanism is **Signals**, which are used to notify processes about events such as termination or errors. **Sockets** allow communication between processes over a network or on the same system. These IPC mechanisms help processes cooperate efficiently and ensure smooth system operation.

21. Explain the CPU Scheduling Algorithm. Write advantages and disadvantages

CPU scheduling is the method used by the operating system to decide which process in the ready queue should be given the CPU next. Since many processes may be waiting for execution, CPU scheduling helps in efficient use of the CPU and improves system performance. The operating system uses different scheduling algorithms based on system requirements such as response time, throughput, and fairness.

Common CPU scheduling algorithms include First Come First Serve (FCFS), Shortest Job First (SJF), Priority Scheduling, and Round Robin. FCFS executes processes in the order they arrive. SJF selects the process with the shortest execution time. Priority scheduling runs the process with the highest priority first. Round Robin assigns each process a fixed time slice in a cyclic order.

Advantages:

CPU scheduling improves CPU utilization and system efficiency. It supports multitasking and provides fair CPU access to processes. Time-sharing systems offer quick response to users.

Disadvantages:

Some algorithms may cause starvation, where certain processes wait too long. Context switching increases overhead. Choosing the best algorithm is complex and depends on system needs.

22. Identify the Critical Section Problem and its requirements

The critical section problem occurs in a multiprogramming or multitasking environment where multiple processes or threads share common resources, such as variables, files, or memory. A critical section is the part of a program where shared resources are accessed or modified. If two or more processes enter their critical sections at the same time, it can lead to data inconsistency and incorrect results.

The critical section problem is concerned with designing a mechanism that ensures correct and safe access to shared resources. The main goal is to allow only one process at a time to execute its critical section.

There are three basic requirements to solve the critical section problem. **Mutual exclusion** ensures that only one process can be in the critical section at any given time. **Progress** means that if no process is in the critical section, the decision of selecting a process to enter should not be postponed indefinitely. **Bounded waiting** ensures that a process does not have to wait forever to enter its critical section and that there is a limit on waiting time.

24. Demonstrate Deadlock prevention and deadlock recovery techniques

Deadlock is a situation in an operating system where two or more processes are permanently blocked because each process is waiting for a resource held by another. To handle deadlock, operating systems use prevention and recovery techniques.

Deadlock prevention works by ensuring that at least one of the four necessary conditions for deadlock never occurs. These conditions are mutual exclusion, hold and wait, no preemption, and circular wait. For prevention, the system may allow a process to request all required resources at once, which avoids hold and wait. Resources can also be preempted from a process if needed. Circular wait can be prevented by assigning resources in a fixed order.

Deadlock recovery techniques are used after a deadlock has already occurred. First, the system detects the deadlock using detection algorithms. Once detected, recovery can be done by terminating one or more deadlocked processes or by preempting resources from selected processes and reallocating them. Although recovery helps resume system operation, it may lead to loss of work or reduced performance.

25. Justify Segmentation with diagrams

Segmentation is a memory management technique in which a program is divided into logical units called segments. Each segment represents a meaningful part of the program such as code, data, stack, or functions. Unlike paging, segmentation is based on the logical view of the programmer, which makes memory management more flexible and understandable.

Segmentation is justified because it supports modular programming. Each segment can be independently protected and shared. For example, the code segment can be marked as read-only, while the data segment can be read and write. It also allows efficient memory use, as segments are created according to the actual size needed, reducing unnecessary memory wastage. Segmentation also makes sharing of common segments, like libraries, easier among multiple processes.

However, segmentation may suffer from external fragmentation because segments are of variable size. Still, its advantages in logical organization, protection, and sharing make it important in operating system design.



26. Identify Virtual Memory and Demand Paging

Virtual memory is a memory management technique that allows a computer to execute programs that are larger than the available physical main memory. It creates an illusion of a large memory by using secondary storage, such as a hard disk, as an extension of main memory. Only the required parts of a program are loaded into main memory, which improves memory utilization and allows multiple programs to run efficiently at the same time.

Demand paging is a method used to implement virtual memory. In demand paging, pages of a program are loaded into main memory only when they are needed during execution. If a program tries to access a page that is not currently in memory, a page fault occurs. The operating system then fetches the required page from secondary storage and loads it into memory.

Virtual memory reduces memory requirements, increases multiprogramming, and improves system performance. Demand paging avoids unnecessary loading of pages, but frequent page faults can reduce performance if not managed properly.

27. Explain Page Replacement Algorithms: FIFO and LRU

Page replacement algorithms are used in an operating system to decide which page should be removed from main memory when a new page needs to be loaded and the memory is full. These algorithms help manage memory efficiently and reduce page faults.

FIFO (First In First Out) is the simplest page replacement algorithm. In FIFO, the page that was loaded into memory first is removed first when a replacement is needed. It follows a queue structure. For example, if pages A, B, and C enter memory in this order, page A will be replaced first, even if it is frequently used. The advantage of FIFO is that it is easy to understand and implement. However, its disadvantage is that it may remove important pages, leading to more page faults.

LRU (Least Recently Used) replaces the page that has not been used for the longest period of time. It considers the recent usage history of pages. For example, among pages A, B, and C, if page B was used least recently, it will be replaced. LRU usually performs better than FIFO but is more complex to implement.

28. Prepare File Allocation Methods and Free-Space Management

File allocation methods define how files are stored on a disk and how disk blocks are assigned to them. One common method is **Contiguous Allocation**, where all blocks of a file are stored in consecutive locations. It is simple and provides fast access, but it causes external fragmentation and makes file expansion difficult.

Another method is **Linked Allocation**, where each file block contains a pointer to the next block. This removes external fragmentation and allows easy file growth, but accessing files is slower and pointers consume extra space.

Indexed Allocation uses an index block that stores addresses of all file blocks. It allows direct access and avoids fragmentation, but requires extra space for the index block.

Free-space management keeps track of unused disk blocks. In the **Bit Vector method**, each block is represented by a bit, where 0 means free and 1 means allocated. It is simple but uses memory. The **Linked List method** links all free blocks together, saving space but slowing access. The **Grouping method** stores addresses of multiple free blocks together, improving efficiency. Efficient file allocation and free-space management help improve disk performance and storage utilization.

29. Manage Disk Scheduling Algorithms and Disk Management

Disk scheduling algorithms are used by the operating system to decide the order in which disk read and write requests are processed. The main goal is to reduce seek time, improve disk performance, and increase system efficiency.

FCFS (First Come First Serve) services disk requests in the order they arrive. It is simple but may result in long waiting time. **SSTF (Shortest Seek Time First)** selects the request closest to the current disk head position, reducing seek time but may cause starvation. **SCAN (Elevator Algorithm)** moves the disk head in one direction, servicing requests along the way, then reverses direction. **C-SCAN** services requests in one direction only, providing more uniform waiting time. **LOOK and C-LOOK** are improved versions that stop at the last request instead of going to the disk end.

Disk management involves organizing and controlling disk storage. It includes disk formatting, partitioning, file system management, and bad block handling. The operating system also manages free disk space and ensures data is stored and accessed efficiently. Proper disk scheduling and management improve system speed, reliability, and overall performance.

30. Explain OS security, Virtual Machines, and Cloud OS concepts

Operating System security refers to the protection mechanisms used by an OS to safeguard data, programs, and system resources from unauthorized access and attacks. OS security includes user authentication, access control, encryption, firewalls, and malware protection. It ensures that only authorized users can access system resources and that data integrity and confidentiality are maintained. Regular updates and patches also help protect the system from security threats.

A Virtual Machine (VM) is a software-based simulation of a physical computer. It allows multiple operating systems to run on a single physical machine at the same time. Each virtual machine works independently with its own OS and applications. Virtual machines improve hardware utilization, provide isolation, and are useful for testing, development, and server consolidation.

A Cloud Operating System is designed to manage and operate resources in a cloud computing environment. It controls virtual machines, storage, networking, and services across multiple servers. Cloud OS supports scalability, flexibility, and remote access to resources through the internet. It enables users to use computing resources on demand without worrying about underlying hardware.

31. Compare Batch OS, Multiprogrammed OS, and Time-sharing OS

A Batch Operating System processes jobs in groups called batches, where similar jobs are collected and executed one after another without any user interaction. Users submit their jobs and wait for the output. CPU utilization is low because the CPU may remain idle during input/output operations, and the turnaround time is high. Batch systems are simple but not suitable for interactive work.

A Multiprogrammed Operating System allows multiple programs to be loaded into memory at the same time. When one process is waiting for input/output, the CPU switches to another process, which improves CPU utilization and overall system efficiency. There is little or no direct interaction with users, but performance is much better than batch operating

systems.

A Time-sharing Operating System allows multiple users to interact with the system simultaneously. Each process is given a small time slice of the CPU in a cyclic manner. This provides fast response time and good user interaction. Time-sharing systems are more complex but are widely used in modern multi-user and interactive environments.

32. Explain the structure of an Operating System with a neat diagram

The structure of an Operating System (OS) shows how its different components are organized and how they interact to manage computer resources efficiently. A well-defined structure helps in better performance, easy maintenance, and reliable system operation.

At the lowest level is the **hardware**, which includes the CPU, main memory, and input/output devices. Directly above the hardware is the **kernel**, which is the core of the operating system. The kernel is responsible for essential functions such as process management, memory management, CPU scheduling, device management, and handling system calls. It controls all system resources and ensures smooth interaction between hardware and software.

Above the kernel are **system programs** or system utilities. These include file management tools, device drivers, and system services that provide a convenient environment for application programs. At the top layer are **application programs**, such as word processors, browsers, and media players, which users directly interact with. Users access the system through a **user interface**, either command-line or graphical.

Diagram:

Users

↓

Application Programs

↓

System Programs

↓

Kernel

↓

Hardware

33. State fragmentation. Differentiation between internal & external fragmentation

Fragmentation is a memory management problem that occurs when available memory is not used efficiently. It happens when memory is divided into small pieces, causing wastage of memory space and difficulty in allocating memory to processes. Even if enough total memory is available, fragmentation can prevent proper allocation and reduce system performance.

Internal fragmentation occurs when a process is allocated more memory than it actually needs. The unused memory lies inside the allocated block and cannot be used by other processes. This usually happens in fixed-size memory allocation. For example, if a process needs 14 KB but is given a 16 KB block, the extra 2 KB is wasted.

External fragmentation occurs when free memory is available in small scattered blocks throughout the memory. Although the total free memory may be sufficient, it is not available as one continuous block. This occurs in variable-size memory allocation systems. External fragmentation can be reduced using techniques like compaction.

In short, internal fragmentation wastes memory within allocated blocks, while external fragmentation wastes memory between allocated blocks.

34. Define evolution of operating system in detail

The evolution of the operating system describes how operating systems developed over time to meet the growing needs of computer users and advancing hardware technology. In the beginning, early computers had no operating system. Programs were loaded manually using switches and punched cards, and only one job could run at a time. This stage is known as serial processing systems.

Next came batch operating systems, where jobs with similar requirements were grouped and executed in batches without user interaction. This improved system utilization but had long waiting times. After that, multiprogramming operating systems were introduced, allowing several programs to reside in memory at the same time. This reduced CPU idle time and improved efficiency.

Time-sharing operating systems were developed to support multiple users interacting with the system simultaneously, each receiving a small time slice of the CPU. With the rise of personal computers, single-user and multitasking operating systems became popular. Modern operating systems now support multiprocessing, multithreading, distributed systems, real-time processing, strong security, networking, graphical interfaces, virtualization, and cloud computing, making them powerful and user-friendly.

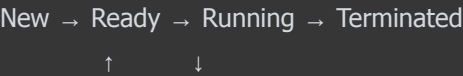
35. Elaborate Process Life Cycle with diagram

The process life cycle describes the different states a process goes through from its creation to its completion in an operating system. It helps the operating system manage and control multiple processes efficiently.

When a program is started, it enters the **New** state, where the process is being created and resources are allocated. After creation, the process moves to the **Ready** state, where it waits in the ready queue for CPU allocation. When the CPU scheduler selects it, the process enters the **Running** state and begins execution.

If the process requires input/output operations or needs to wait for an event, it moves to the **Waiting (or Blocked)** state. Once the event is completed, the process returns to the Ready state. This cycle between Ready, Running, and Waiting may occur several times during execution. When the process finishes execution, it enters the **Terminated** state, and all allocated resources are released.

Diagram:



36. Distinguish between system software and application software

System software is a type of software that controls and manages the basic operations of a computer system. It acts as an interface between the computer hardware and the user. System software is essential for the functioning of a computer and starts running as soon as the system is powered on. It manages hardware resources such as CPU, memory, and input/output devices. Examples of system software include operating systems like Windows and Linux, along with device drivers and system utilities. Users do not directly use system software for their daily tasks.

Application software, on the other hand, is designed to perform specific tasks for users. It depends on system software to run and cannot function without it. Application software helps users complete tasks such as writing documents, browsing the internet, or playing games. Examples include Microsoft Word, spreadsheets, web browsers, media players, and games.

In short, system software manages and controls the computer system, while application software helps users perform specific activities on the computer.

39. Explain different scheduling algorithms with advantage & disadvantage

Scheduling algorithms are used by the operating system to decide which process should get the CPU next. Different algorithms are used based on system requirements like fairness, response time, and efficiency.

First Come First Serve (FCFS) schedules processes in the order they arrive.

Advantage: Simple and easy to implement.

Disadvantage: Long waiting time and poor performance if a long process comes first.

Shortest Job First (SJF) selects the process with the shortest execution time.

Advantage: Minimizes average waiting time.

Disadvantage: Difficult to predict burst time and may cause starvation of long processes.

Priority Scheduling runs the process with the highest priority first.

Advantage: Important processes get CPU quickly.

Disadvantage: Low-priority processes may starve.

Round Robin (RR) assigns each process a fixed time slice in a cyclic order.

Advantage: Fair scheduling and good response time for interactive systems.

Disadvantage: Too small time slice causes overhead; too large makes it like FCFS.

Each algorithm has its own strengths and weaknesses, and the choice depends on system needs.

40. Justify File allocation methods and free-space management

File allocation methods explain how files are stored on a disk and how disk blocks are assigned to them. These methods are important because they affect file access speed, storage efficiency, and system performance. **Contiguous allocation** stores a file in consecutive disk blocks. It is simple and provides fast access, but it causes external fragmentation and makes file growth difficult. **Linked allocation** stores files as linked lists of blocks. It avoids external fragmentation and allows files to grow easily, but access time is slower and extra space is needed for pointers. **Indexed allocation** uses an index block that contains addresses of all file blocks. It supports direct access and reduces fragmentation, but it requires extra storage for the index.

Free-space management keeps track of unused disk space so new files can be stored efficiently. Methods such as the **bit vector** method provide a simple way to track free blocks, while **linked lists** and **grouping** methods improve space management efficiency. Together, proper file allocation and free-space management ensure better disk utilization, faster file access, and reliable storage management in an operating system.

41. Describe threads and multithreading models and its types with diagram

A thread is the smallest unit of CPU execution within a process. A process can have multiple threads that share the same memory, files, and resources, but each thread has its own program counter, registers, and stack. Threads help improve system performance by allowing multiple tasks of a process to run concurrently.

Multithreading is the ability of an operating system to support multiple threads within a single process. It improves CPU utilization, responsiveness, and resource sharing. There are different multithreading models used to map user threads to kernel threads.

In the **Many-to-One model**, many user-level threads are mapped to a single kernel thread. It is easy to manage but does not support parallel execution.

In the **One-to-One model**, each user thread is mapped to a separate kernel thread. It supports true parallelism but has higher overhead.

In the **Many-to-Many model**, many user threads are mapped to a smaller or equal number of kernel threads. It combines efficiency and flexibility.

Diagram:

Many-to-One

U1

U2 → K1

U3

One-to-One

U1 → K1

U2 → K2

U3 → K3

Many-to-Many

U1, U2 → K1

U3, U4 → K2

42. Differentiate between demand paging and page replacement algorithms (FIFO, LRU, OPTIMAL) with examples

Demand paging is a memory management technique used in virtual memory systems where pages are loaded into main memory only when they are required during execution. If a program tries to access a page that is not in memory, a page fault occurs, and the operating system brings that page from secondary storage. For example, if a program has pages A, B, C, and D, and only A and B are needed initially, then only those pages are loaded into memory.

Page replacement algorithms are used when the memory is full and a new page must be loaded. They decide which existing page should be removed. FIFO (First In First Out) replaces the page that entered memory first. For example, if pages A, B, and C are loaded in order, A will be replaced first. LRU (Least Recently Used) replaces the page that has not been used for the longest time, such as removing B if it was accessed least recently. OPTIMAL replaces the page that will not be used for the longest time in the future, giving the best result but is difficult to implement.

In short, demand paging decides *when* to load pages, while page replacement decides *which* page to remove.

44. Discuss Disk scheduling algorithms and disk management

Disk scheduling algorithms are used by the operating system to decide the order in which disk read and write requests are processed. The main objective is to reduce seek time, minimize head movement, and improve overall disk performance.

FCFS (First Come First Serve) processes disk requests in the order they arrive. It is simple but may cause long waiting time. **SSTF (Shortest Seek Time First)** selects the request closest to the current head position, which reduces seek time but may lead to starvation. **SCAN (Elevator Algorithm)** moves the disk head in one direction, servicing requests along the way, and then reverses direction. **C-SCAN** services requests in only one direction, providing uniform waiting time. **LOOK and C-LOOK** are improved versions that stop at the last request instead of going to the disk end.

Disk management involves tasks such as disk formatting, partitioning, file system management, free-space management, and bad block handling. The operating system also ensures data integrity and efficient storage use. Proper disk scheduling and management improve system speed, reliability, and performance.

45. Explain OS security, Virtual Machines, and Cloud OS concepts

Operating System (OS) security refers to the measures used to protect a computer system from unauthorized access, misuse, and attacks. It includes user authentication, access control, encryption, firewalls, and malware protection. OS security ensures data confidentiality, integrity, and availability by allowing only authorized users to access system resources and by preventing malicious activities.

A Virtual Machine (VM) is a software-based imitation of a physical computer. It allows multiple operating systems to run on a single physical machine at the same time. Each virtual machine has its own operating system and applications and works independently from others. Virtual machines help in better hardware utilization, isolation, testing, and running multiple environments on the same system.

A Cloud Operating System is designed to manage computing resources in a cloud environment. It controls virtual machines, storage, networking, and services across many servers. Cloud OS provides scalability, flexibility, and on-demand access to resources over the internet, allowing users to use computing services without managing physical hardware.

46. State Inter-Process Communication (IPC)

Inter-Process Communication (IPC) is a mechanism provided by the operating system that allows different processes to communicate with each other and coordinate their actions. Since processes run independently and have separate memory spaces, IPC offers safe and controlled methods for sharing data and information between them.

IPC is mainly used in multitasking and multiuser systems where several processes execute at the same time. It helps processes cooperate, synchronize, and avoid conflicts while accessing shared resources. Common IPC mechanisms include pipes, message queues, shared memory, semaphores, signals, and sockets. Pipes allow one process to send data to another. Message queues support structured message exchange. Shared memory provides fast communication by allowing processes to access the same memory area, while semaphores help in synchronization. Signals notify processes about events, and sockets enable communication over a network. IPC improves system efficiency and reliability.

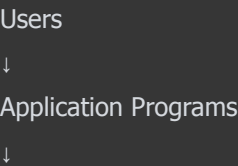
47. Describe the structure of an Operating System with a neat diagram

The structure of an Operating System (OS) explains how its components are organized and how they interact to manage hardware and software resources efficiently. A well-structured OS improves performance, reliability, and ease of maintenance.

At the lowest level is the **hardware**, which includes the CPU, main memory, and input/output devices. Directly above the hardware is the **kernel**, which is the core of the operating system. The kernel is responsible for critical functions such as process management, CPU scheduling, memory management, device management, and handling system calls. It controls access to hardware and ensures safe and efficient resource use.

Above the kernel are **system programs** or system utilities. These provide services such as file management, device drivers, and system tools that support application programs. At the top layer are **application programs** like text editors, browsers, and media players, which users directly interact with. Users access the system through a **user interface**, either command-line or graphical.

Diagram:



System Programs

↓

Kernel

↓

Hardware

48. Define fragmentation. Differentiation between internal & external fragmentation

Fragmentation is a memory management problem that occurs when available memory is not used efficiently. It results in wastage of memory space and difficulty in allocating memory to processes, even when enough total memory exists. Fragmentation reduces system performance and effective memory utilization.

Internal fragmentation occurs when a process is allocated more memory than it actually requires. The unused memory lies inside the allocated block and cannot be used by other processes. This usually happens in fixed-size memory allocation systems. For example, if a process needs 10 KB but is given a 12 KB block, the extra 2 KB remains unused.

External fragmentation occurs when free memory is divided into small, scattered blocks throughout the memory. Although the total free memory may be sufficient, it is not available as one continuous block to allocate to a process. This occurs in variable-size memory allocation systems. External fragmentation can be reduced using techniques like compaction.

In short, internal fragmentation wastes memory within allocated blocks, while external fragmentation wastes memory between allocated blocks.

49. Express various Operating System services

Operating System services are the facilities provided by the operating system to make computer usage convenient, efficient, and secure. These services help users and application programs interact with the hardware properly.

One important service is **program execution**, which loads programs into memory, runs them, and terminates them after execution. **Process management** allows the OS to create, schedule, and control processes, enabling multitasking. **Memory management** allocates memory to programs and frees it when no longer needed.

The OS also provides **file system services** to create, delete, read, write, and organize files on storage devices. **Input/Output device management** controls communication between hardware devices and programs using device drivers. **Communication services** allow processes to exchange data using mechanisms like IPC and networking.

For safety, **security and protection services** ensure authentication and control access to resources. **Error detection and handling** identify and manage system errors. Finally, the OS provides a **user interface**, such as a command-line or graphical interface, for easy interaction with the computer system.

50. What is Segmentation?

Segmentation is a memory management technique used in operating systems in which a program is divided into logical units called segments. Each segment represents a meaningful part of a program such as code, data, stack, or functions. Unlike paging, segmentation is based on the logical structure of a program, which makes it easier for programmers to understand and manage memory.

In segmentation, each segment can be of different size depending on the program's requirement. The operating system maintains a segment table that stores information about each segment, such as its base address and limit. A logical address in segmentation consists of a segment number and an offset. Using the segment table, the OS converts this logical address into a physical address.

Segmentation provides advantages like better memory protection, easier sharing of code and data, and support for modular programming. However, since segments are of variable size, segmentation can suffer from external fragmentation. Despite this drawback, segmentation is important because it reflects the logical view of a program and improves memory organization.

51. Differentiate between process & thread

A process is a program that is currently being executed. It has its own memory space, system resources, and execution environment. Processes are independent of each other, which means one process cannot directly access the memory of another process. Creating and managing a process requires more system resources and time. Communication between processes is slower and usually done using inter-process communication mechanisms.

A thread is the smallest unit of execution within a process. Multiple threads can exist within the same process and share the same memory space, files, and resources. Threads are lightweight and require less time and fewer resources to create and manage compared to processes. Communication between threads is faster because they share memory, but it also requires careful synchronization to avoid errors.

In simple terms, a process is a complete program in execution, while a thread is a part of that program. Processes provide better isolation and security, whereas threads provide better efficiency and faster execution.

53. Elaborate the CPU Scheduling Algorithm. Write advantages and disadvantages

CPU scheduling is the technique used by an operating system to decide which process should be assigned the CPU when multiple processes are ready for execution. Since the CPU is a limited resource, proper scheduling is necessary to improve system performance, responsiveness, and fairness.

Several CPU scheduling algorithms are used in operating systems. **First Come First Serve (FCFS)** schedules processes in the order they arrive. **Shortest Job First (SJF)** selects the process with the shortest CPU burst time. **Priority Scheduling** executes processes based on priority levels. **Round Robin (RR)** gives each process a fixed time slice in a circular manner, making it suitable for time-sharing systems.

Advantages:

CPU scheduling improves CPU utilization and system efficiency. It supports multitasking and provides fair access to processes. Time-sharing algorithms offer quick response to users

and improve interactivity.

Disadvantages:

Some algorithms may cause starvation of certain processes. Context switching introduces overhead. Selecting the most suitable scheduling algorithm is complex and depends on system requirements such as response time and throughput.

54. Prepare File allocation methods and free-space management

File allocation methods describe how files are stored on a disk and how disk blocks are assigned to each file. One method is **Contiguous Allocation**, where all blocks of a file are stored in continuous disk locations. It is simple and provides fast access, but it causes external fragmentation and makes file expansion difficult.

Linked Allocation stores file blocks at different locations, where each block points to the next one. It avoids external fragmentation and allows files to grow easily, but access is slower and extra space is required for pointers.

Indexed Allocation uses a special index block that contains addresses of all file blocks. It supports direct access and avoids fragmentation, but additional space is needed for the index block.

Free-space management keeps track of unused disk blocks. The **Bit Vector method** uses bits to represent free and allocated blocks. The **Linked List method** links all free blocks together, saving space but slowing access. **Grouping** and **counting methods** improve efficiency by managing free blocks in groups.

Together, proper file allocation and free-space management ensure efficient disk usage, faster file access, and better overall system performance.

55. Describe demand paging and page replacement algorithms (FIFO, LRU, OPTIMAL) with examples

Demand paging is a memory management technique used in virtual memory systems where pages of a program are loaded into main memory only when they are needed. If a program accesses a page that is not in memory, a page fault occurs, and the operating system brings that page from secondary storage. For example, if a program has pages A, B, C, and D, but only A and B are used initially, then only those pages are loaded into memory, saving space.

When memory is full and a new page must be loaded, page replacement algorithms decide which page to remove. FIFO (First In First Out) replaces the page that was loaded first. For example, if pages A, B, and C entered memory in that order, page A will be removed first. LRU (Least Recently Used) removes the page that has not been used for the longest time, such as removing page B if it was accessed least recently. OPTIMAL replaces the page that will not be used for the longest time in the future. It gives the best performance but is not practical to implement.

Demand paging decides *when* to load pages, while page replacement decides *which* page to remove.

58. Examine directory implementation method and file allocation method (linked, indexed, contiguous)

Directory implementation methods define how directories store information about files. A **single-level directory** contains all files in one directory, which is simple but causes name conflicts. A **two-level directory** separates user files into different directories, reducing conflicts. A **tree-structured directory** allows directories within directories, forming a hierarchy that is easy to organize and search. A **graph-structured directory** allows sharing of files and directories using links but is more complex to manage.

File allocation methods describe how disk blocks are assigned to files. In **contiguous allocation**, all blocks of a file are stored in consecutive locations. It provides fast access and simple management but causes external fragmentation and makes file expansion difficult.

In **linked allocation**, file blocks are stored at different locations and linked using pointers. It avoids external fragmentation and supports file growth, but access time is slow and pointer overhead is high.

In **indexed allocation**, an index block stores the addresses of all file blocks. It supports direct access and avoids fragmentation, but extra space is needed for the index block.

Each method has advantages and limitations, and operating systems choose them based on performance and storage requirements.

59. Explain OS security, Virtual Machines, and Cloud OS concepts

Operating System (OS) security refers to the techniques and mechanisms used to protect a computer system from unauthorized access, misuse, and threats. It includes user authentication, access control, encryption, firewalls, and protection against malware. OS security ensures data confidentiality, integrity, and availability by allowing only authorized users and programs to access system resources. Regular updates and security policies also help reduce vulnerabilities.

A Virtual Machine (VM) is a software-based environment that behaves like a physical computer. It allows multiple operating systems to run on a single physical machine at the same time. Each virtual machine has its own operating system, applications, and virtual hardware. Virtual machines improve hardware utilization, provide isolation between systems, and are widely used for testing, development, and server consolidation.

A Cloud Operating System is designed to manage computing resources in a cloud environment. It controls virtual machines, storage, networking, and services across multiple servers. Cloud OS enables scalability, flexibility, and on-demand access to resources through the internet, allowing users to use computing services without managing physical hardware.

60. Demonstrate Deadlock prevention and deadlock recovery techniques

Deadlock is a condition in an operating system where two or more processes are permanently blocked because each process is waiting for a resource held by another process. To handle deadlocks, operating systems use prevention and recovery techniques.

Deadlock prevention aims to ensure that at least one of the four necessary conditions for deadlock never occurs. These conditions are mutual exclusion, hold and wait, no preemption, and circular wait. Prevention techniques include allowing a process to request all required resources at once to avoid hold and wait, preempting resources from processes when needed, and allocating resources in a fixed order to prevent circular wait.

Deadlock recovery is applied after a deadlock has already occurred. First, the system detects the deadlock using detection algorithms. Recovery can then be done by terminating one or more deadlocked processes or by preempting resources from selected processes and reallocating them. Although recovery resolves deadlock, it may result in loss of work and reduced system performance.